Indian Sign Language Gesture Recognizer

Understanding guide - in simple language

1. The problem, in one paragraph

Many people communicate with sign language, but most computers only understand typing and speech. This project is a first step: a webcam watches a hand, and the computer names the gesture being signed, in real time. It is a demo, not a finished translator. It knows only 8 gestures: hello, thank you, yes, no, please, help, sorry and bye.

2. How hand landmarks work

Instead of trying to understand a whole video picture, the project uses MediaPipe Hands, a ready-made tool that finds a hand in each video frame and returns 21 points, called landmarks: the wrist, the knuckles, and the joints and tips of all five fingers. Each point has three numbers: x (left-right), y (up-down) and z (how far forward or back). So one frame of one hand is 21 points x 3 numbers = 63 numbers. Landmarks are much easier to learn from than raw pixels, because the computer no longer has to learn what a hand looks like - it is given the hand's shape directly.

One gesture is not one picture, it is a movement. So the project collects 30 frames in a row, about one second of video. One full gesture is therefore 30 frames x 63 numbers = 1,890 numbers, in order, so the movement over time is kept in the data.

3. Step-by-step workflow

Step 1. Collect: run collect_data.py, press S, and perform one gesture steadily for about a second. It saves one file per performance, shaped (30, 21, 3). Record at least 20 samples of each of the 8 gestures.

Step 2. Clean: every frame is centred on the wrist and divided by hand size (see normalize_frame). This removes the effect of where the hand is in the frame and how close it is to the camera.

Step 3. Flatten: each 30-frame gesture becomes one row of 1,890 numbers (see sequence_to_features).

Step 4. Train: train.py splits the data 80/20, keeping every gesture in both parts, and trains a RandomForest classifier - 300 decision trees that vote. No GPU is needed.

Step 5. Evaluate honestly: accuracy is measured only on the 20% the model never trained on. The confusion matrix shows which gestures get mixed up, and per-class F1 shows the score for each gesture separately.

Step 6. Demo: demo.py watches the webcam, keeps the last 30 frames, predicts every frame, and shows the majority vote of the last 5 predictions, so the label does not flicker.

Step 7. Improve: if accuracy is low, record more samples, in different lighting, and from more than one person - then retrain and compare.

4. Every main function: what it does, why, and what it outputs

common.normalize_frame(landmarks)

Subtracts the wrist point and divides by hand size. Why: so position and distance do not confuse the model. Output: a (21, 3) array with the wrist at zero.

common.sequence_to_features(sequence)

Normalises all 30 frames and flattens them in order. Why: the classifier needs one fixed-size row per gesture. Output: a (1890,) number vector.

common.features_to_sequence(features)

The reverse of the above, for checking and debugging only. Output: a (30, 21, 3) array.

generate_sample_data._base_hand(pose)

Builds a simple flat-hand skeleton from five finger-openness values. Why: to give each synthetic gesture a distinct shape. Output: a (21, 3) hand.

generate_sample_data._motion(gesture, frame)

Returns the small x/y/z shift for one frame of one gesture, for example hello waves side to side and yes bobs down. Output: a 3-number shift.

generate_sample_data.generate_sample_data(samples, seed)

Makes synthetic gestures with noise, scale and shift, for pipeline testing only. Output: X shaped (n, 30, 21, 3) and labels y.

collect_data.extract_landmarks(results)

Takes the first detected hand from a MediaPipe result. Output: a (21, 3) array, or None when no hand is visible.

collect_data.record_sample(cap, hands)

Counts down 3-2-1, then records 30 frames, filling brief hand dropouts with the last good frame. Output: a (30, 21, 3) sample, or None if the hand was mostly lost.

train.load_real_data(data_dir)

Loads .npy files from data/<gesture>/ folders. Output: (X, y), or (None, None) when no real data exists, so training can fall back to synthetic data.

train.load_sample_data(path)

Loads the synthetic .npz file. Output: (X, y) synthetic landmarks and labels.

train.build_model()

Builds StandardScaler plus RandomForest with 300 trees. Why: fast on CPU, robust on a few hundred samples, no GPU needed. Output: a scikit-learn pipeline.

train.evaluate(model, X_test, y_test)

Predicts on held-out data and computes accuracy, per-class precision/recall/F1 and the confusion matrix. Output: a dictionary of metrics.

train.plot_results(metrics, results_dir)

Saves the confusion matrix chart and the per-class F1 chart. Output: two PNG files under results/.

demo.smooth_predictions(history)

Returns the most common label among the last 5 predictions. Why: it stops single-frame flicker. Output: one gesture name, or None.

Each script's main()

Handles command-line arguments and runs that script end to end: collect_data records and saves, train trains/evaluates/saves, demo runs the live webcam loop.

5. Results - and the honest caveat

On the synthetic sample data (480 samples, 60 per gesture), the held-out accuracy was 1.0000, that is 96 out of 96 test samples correct, with F1 of 1.00 for every gesture. Do NOT quote this as real-gesture accuracy. The synthetic gestures were deliberately built with different shapes and motions, so a perfect score mainly proves the pipeline works end to end. Real accuracy must be measured after retraining on real recorded samples, on held-out real samples, and reported with that label.

6. Limits to say out loud in an interview

Only 8 gestures; anything else is forced into one of those labels. Only one hand, and only one signer if trained on one

signer. Needs good lighting and the full hand in frame. Real ISL also uses two hands, facial expression and grammar, which this demo does not model. The webcam scripts were not testable in the build environment, so they still need a live webcam test.

7. Your 60-second interview script

"I built a demo that recognises 8 Indian Sign Language gestures from a webcam. MediaPipe gives me 21 hand landmarks per frame, I take 30 frames per gesture, normalise each frame by the wrist and hand size, and flatten it to 1,890 features. I train a RandomForest classifier on that, evaluate on a held-out split with a confusion matrix, and the live demo smooths predictions over the last five frames so the label does not flicker. Honestly, my 100 percent score is on clearly labelled synthetic data, which proves the pipeline works - the next step is recording real samples and reporting real held-out accuracy. The limits are a small vocabulary, one hand, and no facial expression or grammar yet."

8. Five likely interview questions and answers

Q1. Why landmarks instead of raw images?

Raw images force the model to learn what a hand looks like from pixels, which needs far more data and computing. Landmarks give the hand's shape directly as 63 numbers per frame, so a small dataset and a CPU classifier are enough.

Q2. Why normalise by the wrist and hand size?

Without it, the model learns where the hand was in the frame, or how close it was to the camera, instead of the gesture itself. Centring on the wrist and scaling by hand size removes position and size, leaving shape and motion.

Q3. Why a RandomForest and not a deep neural network or LSTM?

An LSTM models time more directly, but needs more data, TensorFlow, and longer training. With a few hundred samples, a RandomForest on flattened sequences trains in seconds on CPU and is robust. The code keeps build_model() as the one place to swap in an LSTM later.

Q4. Your accuracy is 100 percent - is that believable?

No, not as real-gesture accuracy, and I would not present it that way. It is on synthetic data with deliberately separable classes, so it proves the pipeline runs end to end. Real accuracy comes only from held-out real recordings.

Q5. How would you improve this into something more real?

First, record real data from several signers in varied lighting and retrain. Then add an 'unknown' class so unseen gestures are not forced into the 8 labels, try a sequence model like an LSTM, add two-hand support, and check the handshapes against a proper ISL resource.